Containers for Typed Agentic AI

Professor Neil Ghani
Co-founder & Chief Scientific Officer, Kodamai

Agentic AI systems are built by wiring together components that each take a request and produce a response: a router here, a tool call there, a language model at the end. The pieces are usually assembled by hand, with untyped text flowing between them and little to guarantee that what one component emits is something the next can actually use. This note argues that there is a single mathematical object underneath all of it — the *container* — and that taking it seriously lets the same type discipline that protects ordinary software protect a system of agents.

The picture to keep in mind is a hierarchy of abstraction. Think of a project manager and a software developer: the manager works with a high-level requirement — "handle a customer refund" — and delegates to the developer, who works with lower-level operations — "read the order, check the payment, issue the credit" — whose replies flow back up to answer the original requirement. An agentic system has exactly this shape, often many levels deep. Types formalise what a piece of work must deliver; dependent types let those specifications say what actually matters; containers specify the prompt-and-response interfaces at each level; and *event handlers* (container morphisms) are precisely the act of one level delegating to the next. We build this up in stages, show how the standard agent *workflows* are exactly the ways of combining containers, and end by locating the large language model itself: an untyped oracle sitting at the leaves of a typed tree of interfaces.

It is worth saying at the outset why this is worth your time. A container is an almost embarrassingly simple idea — a set of prompts, and for each prompt a set of allowed replies — and yet, as we will see, that one idea reaches from the smallest questions of program correctness all the way up to the architecture of agentic systems. It is genuinely remarkable how far such a modest starting point travels: the same handful of constructions reappear as design patterns practitioners discovered independently, and, once or twice, the mathematics even points ahead to patterns not yet in common use. Simple ideas, taken seriously, have a habit of paying out far more than they cost.

In a nutshell

Strong typing gives ordinary software development a suite of advantages: trust, scalability, governance, correct-by-construction programming, and genuine mathematical guarantees. Containers extend *all* of these to event-driven programming — and therefore to agentic AI. The interfaces that agents present, the ways they hand work off, and the workflows that combine them are exactly the containers, their morphisms, and their monoidal structure; so everything the type checker does for a normal program, it can do for a system of agents.

Structure of this note

- **§1, Strongly typed programming.** Why types are worth having: a type is a machine-checked specification, and from that one idea come trust, scalability, adaptability, and the goal of correct-by-construction programming.
- **§2, Dependent types.** Types that depend on values, letting a specification say as little or as much as you like — from "a list" up to "a provably sorted permutation." Introduces the running bookkeeping example via double-entry as a type.
- **§3, Event-driven programming.** The *container*: a type of prompts and, for each, a type of valid responses. The programmer's job is to turn a prompt into a valid response.
- **§4, Event handlers.** How one interface is implemented by delegating to a lower-level one — container morphisms — and why answering directly is the special case of delegating to nobody.

- **§5, Workflows.** The four ways of combining containers — sequential composition, sum, tensor, and product — and how they match (and, in one case, predict) the standard agent workflow patterns.
- **§6, Where the LLM sits.** The language model as an untyped oracle at the leaves of a typed tree of interfaces: structure from the types, substance from the model.
- **Appendix A** gives the categorical account for the categorically literate reader; **Appendix B** collects the running bookkeeping example into one complete listing.

1 Strongly typed programming

Let us start from the very beginning, because everything later is built on this one idea.

When you write a program, you are describing a computation: take some inputs, do some work, produce some outputs. A *type* is a promise about what those inputs and outputs are. `Int` is the promise "this value is a whole number." `String` is the promise "this value is a piece of text." `List Int` is the promise "this value is a list of whole numbers." A type does not tell you *which* number or *which* text — it tells you what *kind* of thing you have, and therefore what you are allowed to do with it.

Here is the smallest example that shows why this is useful. Suppose you write

```
"apple" + 5
```

What should this mean? You are trying to add the number 5 to the word "apple." There is no sensible answer. In a language without types, this mistake might slip through and cause a crash — or worse, a silent wrong result — only when that line of code happens to run, perhaps months later, in front of a customer. In a *statically typed* language, the program is checked *before it runs*. The checker looks at `"apple"`, sees it is a `String`; looks at `5`, sees it is an `Int`; looks at `+`, which expects two numbers; and reports an error immediately:

```
Error:  can't add a String and an Int.
```

The mistake never makes it into a running program. This is the whole point in miniature: **a type checker catches a class of errors at compile time, before any user is affected.**

Pause on that compile-time/run-time distinction for a moment, because it is easy to undersell. We have grown so used to the compiler catching our mistakes before we ship that we scarcely register what a strange and powerful thing it is: a second program, reading our program, *proving* — not testing, not sampling, but proving — that a whole category of errors is absent, in every execution, before a single line has run. This quiet activity is by now so deeply woven into how we work that we forget it was ever a paradigm shift at all. Much of what follows is really the question: how much more of "it is correct" can we move from run time, where we hope, to compile time, where we know?

Haskell is the language most associated with taking this idea seriously; it is the canonical statically typed functional language, and programmers who use it often repeat the slogan "if it compiles, it works." That is an exaggeration — a program can compile and still do the wrong thing — but it points at something real: once the types are right, a startling number of bugs are simply *impossible to write down*. You cannot pass a `String` where an `Int` is required; you cannot forget to handle a case the type says exists; you cannot return the wrong shape of data. Each of these would be an error a runtime might discover, and a type checker refuses in advance.

The one idea underneath all of this

Why does this help so much? The reason is always the same, and it is worth stating slowly and remembering:

The one idea underneath everything

A type is a formalisation of a *specification*. The specification says what the code is supposed to do; the type expresses that as a precise mathematical object; and the type checker verifies, mechanically, that the code actually has that type — that is, that the code meets its specification.

A *specification* is just a statement of what a piece of software should do: "this function takes a list of orders and returns their total," "this value is never empty," "this input is a valid email address." Normally specifications live in documentation, in comments, in a ticket, or only in someone's head — and nothing checks that the code obeys them. A type takes (part of) that specification and turns it into something the compiler can check on every single build. Three consequences follow, and they are the reasons teams reach for types as their systems grow.

Trust. Because the check is mechanical and runs on every build, you are not trusting a comment that may be out of date, or a test that happens to exercise one path, or a colleague's memory of how the code works — you are trusting a proof. When the compiler accepts your program, it has verified, over the *whole* codebase at once, that the typed part of the specification holds. Our `"apple" + 5` never reaches a customer because it never compiles. That is a guarantee no amount of manual testing can match, because testing checks the cases you thought to try, while the type checker checks all of them.

Scalability. A type checker verifies a million lines as faithfully as it verifies ten. As a system grows, the interfaces between its parts become typed contracts: module A promises to hand module B a `List Order`, and the checker enforces that promise at the boundary. Suppose someone changes A so it sometimes returns nothing at all. Without types, this surfaces as a mysterious failure deep inside B, in production, long after the change. With types, the contract no longer matches, and the compiler points at the exact boundary that broke — the moment the change is made, not the moment a user trips over it. This is how large codebases, worked on by many people, stay coherent.

Adaptability. Software is not written once; it is changed constantly. When you need to alter how something works, the type checker becomes a to-do list. Change an implementation, recompile, and the checker shows you every place that no longer fits the specification — every call site that must be updated, every case you have not yet handled. You can refactor a large system with confidence, because you are not hoping you found all the affected places; the compiler tells you. The type is a scaffold that reveals exactly what your change disturbed.

In all three cases the mechanism is identical: the type states *what* must hold, and the checker confirms the code delivers it.

The aim: correct by construction

Trust, scalability, and adaptability are the day-to-day payoffs. They point at a single, more ambitious goal that gives the whole enterprise its name:

Correct by construction

Make the type a rich enough specification that *any* program which type-checks is already correct. There is then no separate "now test it" or "now prove it" phase: correctness is a *consequence* of the program being well-typed. You do not build the thing and then check it

— you set up the types so that the only programs you *can* build are the correct ones.

This is the ideal that the three payoffs serve: the more of a specification you can push into the type, the more of your correctness comes for free from the checker, and the more trust, scalability, and adaptability you get. It is worth being honest that ordinary types only reach part of the way — a type like `List Int -> List Int` pins down the *shapes* of the input and output, but not that the output is, say, correctly sorted. To make "correct by construction" really bite, we need types expressive enough to state the properties we actually care about. That is the subject of the next section, and it is why the rest of this note is about making the specification language powerful enough to say the things we really mean.

2 Dependent types

The types we have seen so far are, in a sense, *static*. The type `List Int` tells you "a list of whole numbers," but it says nothing about *how many*. `Int` tells you "a whole number," but not *which range*. This is often fine, but it means many specifications we care about cannot be written as types at all — and so cannot be checked.

Let us see the limitation concretely. Consider the function that returns the first element of a list, `head`. What is its type in an ordinary language?

```
head : List a -> a
```

"Give me a list of a's, I give you back an a." But what if the list is empty? There is no first element to return. In Haskell, `head []` is a runtime error — the program crashes when that line runs. The real specification is "*head takes a non-empty list* and returns its first element," but the type `List a -> a` cannot say "non-empty." That part of the specification has leaked out of the type and back into documentation and hope.

Types that depend on values

A **dependent type** is a type that is allowed to depend on a *value*. Formally, it is just a function

```
B : A -> Type
```

which reads: for each value `a` of type `A`, we get a type `B a`. (Here `Type` is the type *of types* — so `B` is a function that takes a value and returns a type. Programmers who have met a language with a kind or a universe of types will recognise it.) That is the entire idea. Everything else is consequences.

The example to hold in your head is the **length-indexed vector**. Instead of `List a`, which forgets the length, we use

```
Vec : Nat -> Type -> Type
```

where `Vec n a` is the type of lists holding *exactly* `n` elements of type `a`. So `Vec 0 a` is the type of empty vectors, `Vec 3 Int` is the type of triples of integers, and a value of type `Vec n a` carries its length *in its type*, where the compiler can see it. The length has been promoted from a runtime fact to a compile-time one. (The element type `a` rides along unchanged; it is the *length* `n` that the dependent type is tracking.)

Watch what this buys us. Here are two specifications that an ordinary type system cannot express, but a dependent one can.

Example 1: a `tail` that refuses empty lists. The function `tail` drops the first element. It

only makes sense on a non-empty list. With vectors we can *say* that in the type:

```
tail : Vec (n + 1) a -> Vec n a
```

Read it slowly: the input type is `Vec (n + 1) a` — a vector whose length is "one more than something," which is precisely a way of saying *non-empty*; and the output is `Vec n a`, one shorter. Now if you try to call `tail` on a vector that might be empty, the lengths do not match and **the program does not compile**. The specification "the input must be non-empty" has become part of the type, so the bug `tail []` is not caught by a test — it cannot be written down at all.

Example 2: a sort that is provably sorted. An ordinary sorting function on integers has type

```
sort : List Int -> List Int
```

which promises only that the output is *a list of integers*. It does not promise the output is sorted, nor that it contains the same elements as the input — those crucial facts are on trust, checked at best by a handful of tests. With dependent types we can make the output type say "a sorted list that is a permutation of the input," so the type becomes something like

```
sort : (xs : List Int) -> SortedPermutationOf xs
```

Now the compiler will reject any implementation that returns a list which is not sorted, or which loses or invents an element. A sorting routine that accidentally drops duplicates does not merely fail a test — it fails to compile. The correctness *is* the type.

Correct by construction, delivered

These two examples are exactly the "correct by construction" aim of §1 finally within reach. There we could only state the goal — types rich enough that well-typed implies correct — and admit that ordinary types fall short: a `List Int -> List Int` cannot forbid a wrong sort. Dependent types close that gap, and they do so in a way that is worth stating as the real headline of this section.

Dial the specification from minimal to exhaustive

The power of dependent types is not a single new feature but a whole *spectrum*. With them you can make a type say as little or as much as you like, and the correct-by-construction guarantee scales with it. Take sorting a list of integers:

- `sort : List Int -> List Int` — the minimal spec: the output is *a list of integers*. Nothing about order.
- `sort : (xs : List Int) -> SameLength xs` — a little more: the output has the same *length* as the input.
- `sort : (xs : List Int) -> Sorted xs` — more still: the output is provably *sorted*.
- `sort : (xs : List Int) -> SortedPermutationOf xs` — the full spec: sorted *and* a permutation of the input, so nothing is lost or invented.

Each rung moves more of "it is correct" from a test you *hope* passes to a type the compiler *checks*. In an ordinary type system only the first rung can be written — you literally cannot express `Sorted xs`, so you cannot have the checker prove your sort is sorted. Dependent types let you climb as high as the property you actually care about, and stop wherever the payoff is worth the effort.

So `tail : Vec (n + 1) a -> Vec n a` makes non-emptiness a compile-time requirement, and a sorted-permutation return type makes sortedness one; in each case the property has moved into the type, so a program that violates it cannot be written down. That is correct by construction

with teeth — and, crucially, teeth you can sharpen incrementally, spec by spec, as far as a given task deserves.

We will develop one example through the rest of this note — a small **bookkeeping assistant** — and accounting gives a perfect first illustration of "dial the specification." The oldest correctness rule in the subject is *double entry*: in every journal entry, total debits must equal total credits. That is exactly a dependent type. Where a loose spec accepts any list of postings, a tighter one demands a proof that they balance:

```
Entry         = List Posting                          -- any postings at all
Balanced ps   = (netAmount ps = 0)                    -- debits = credits, as a
    type
BalancedEntry = (ps : List Posting ** Balanced ps)    -- postings PLUS a proof
```

A `BalancedEntry` carries a proof it balances, so an entry that does not balance is not a runtime error to be caught later — it cannot be constructed at all. The bookkeeper's golden rule has become a type the compiler enforces. We will keep meeting this assistant as each new idea arrives.

A little syntax, introduced gently

Dependent types require only a small extension to notation you already use, and it comes in two flavours: pairs and functions.

An ordinary *pair type* `(A, B)` describes a value that is an `a` together with a `b`, where the type `B` is fixed in advance. A **dependent pair** lets the type of the second component depend on the *value* of the first:

```
(a : A ** B a)
```

Read: "a value `a` of type `A`, together with a value of type `B a`" — and `B a` may be different for different `a`. For instance, "a number `n`, together with a vector of that length" is `(n : Nat ** Vec n Int)`.

An ordinary *function type* `A -> B` describes a function taking an `a` and returning a `b` of a fixed type. A **dependent function** lets the *result* type depend on the argument:

```
(a : A) -> B a
```

Read: "give me an `a`, and I return a value of type `B a`" — so the type of what comes out depends on what you put in.

When `B` happens *not* to depend on `a`, these collapse back to the familiar pair `(A, B)` and function `A -> B`. Dependency is the only new ingredient; if you ignore it you have exactly ordinary pairs and functions.

The same idea, in two dialects

You will also meet these written another way. The dependent pair `(a : A ** B a)` is written $\Sigma\, A\, B$, and the dependent function `(a : A) -> B a` is written $\Pi\, A\, B$. The Σ ("sum") and Π ("product") are worth recognising, because they appear everywhere in the literature — but they name exactly the dependent pairs and functions above, in a different costume.

Why we want this

The specifications you can formalise in a simple, non-dependent type system are genuinely limited. You can say "a list," "a function from integers to integers," "a pair of a string and a boolean." You *cannot* say "non-empty," "sorted," "of length n," or — the one we are heading towards — "a valid response to *this particular* request." Every one of those is a specification that depends on a value, and so needs a dependent type. We want that expressive power, because the interfaces we care about as programmers are full of such value-dependent constraints. The rest of this note is about one especially common and useful shape of dependent specification.

3 Event-driven programming

A great deal of real software is not a single "compute this answer and stop" calculation. Instead it is a standing readiness to *respond*: something happens — an event, a request, a message — and the program must produce an appropriate reaction. A web server waits for requests. A user interface waits for clicks. An agent waits for a prompt. This style is called **event-driven programming**, and it has a feature that turns out to be exactly a dependent type.

The feature is this: *which* responses are appropriate depends on *which* event arrived. The valid answers to "what is your name?" are not the valid answers to "how old are you?" The legal moves in one board position are not the legal moves in another. The response depends on the event — and "a type that depends on a value" is precisely §2.

This pattern is astonishingly common. Here are four instances; notice they all have the same shape.

- A **question** and the **answers** that make sense for it. The question "what year is it?" admits a number; "are you awake?" admits yes or no. Different questions, different admissible answers.
- A **theorem** and its **proofs**. Each statement has its own collection of things that count as a proof of *that* statement; a proof of one theorem is not a proof of another.
- A **data structure** and its valid **pointers** — the positions you are allowed to navigate to. A binary tree of five nodes has five positions; a different tree has different positions.
- A **prompt** and its possible **responses**. This is the case that will matter for language models and agents: a prompt comes in, and there is a space of responses that count as valid replies to it.

Containers

Each of these is modelled by the same object, called a **container**. A container is two pieces of data:

- a type of *prompts* `P : Type` — the events that might occur;
- for each prompt, a type of *responses* `R : P -> Type` — where `R p` is the type of replies that are valid *for the prompt* `p`.

That second line is a dependent type, exactly as in §2: the response type `R p` depends on the prompt `p`. The pair `(P, R)` is a complete *specification of an interface*: it lists every prompt that can arrive, and for each one, exactly which replies are acceptable. Nothing more, nothing less.

Two lines — a type of prompts, and a prompt-indexed type of responses — and we have captured questions-and-answers, theorems-and-proofs, structures-and-pointers, and prompts-and-responses all at once. It is worth noticing how much fell out of so little. Objects this frugal, that nonetheless unify things which look unrelated, are exactly the ones that tend to keep repaying attention; the rest of this note is a series of such dividends.

Let us make it concrete.

Suppose the prompts are two questions. The container records, for each, what counts as a valid answer:

```
P                       = { "what year is it?", "are you awake?" }
R "what year is it?"    = Int       -- an answer is a number
R "are you awake?"      = Bool      -- an answer is yes/no
```

So if asked the year, a valid response is an integer; if asked whether you are awake, a valid response is a boolean. Answering the year with `True` is not merely wrong — it does not even have the right type.

That toy shows the shape. Now our running example gets its interface.

The assistant receives accounting tasks and returns their results. Its interface is the container `Books = (Request, Result)`:

```
data Request = Reconcile    Period      -- reconcile the books for a period
             | RaiseInvoice Customer    -- raise an invoice for a customer
             | TaxEstimate  Year        -- estimate tax for a year

Result : Request -> Type                -- the response type depends on the request
Result (Reconcile p)    = ReconReport   -- matched & unmatched items
Result (RaiseInvoice c) = ValidInvoice  -- a balanced invoice (see section 2)
Result (TaxEstimate y)  = TaxFigure     -- a figure with its breakdown
```

The dependent response type earns its keep: a reconciliation must return a report, an invoice request a validated invoice, a tax query a figure. You cannot answer a `Reconcile` with a `ValidInvoice` — the types do not match, so the mix-up cannot be written down.

The programmer's job

Now we can state, very precisely, what it means to *implement* such an interface.

The programmer's job

Given a container `(P, R)`, write a program of type

```
(p : P) -> R p
```

It is a *dependent function*: it takes a prompt `p` and returns a response whose type is `R p` — the type of valid responses *to that specific prompt*. If it type-checks, every prompt is answered with a genuinely valid response.

Look carefully at that type using §2. A program of this type cannot answer the year with a boolean, because for `p = "what year is it?"` the required output type is `Int`. The container is the specification; the dependent function is the correct-by-construction implementation.

For our little example, an implementation is:

```
answer : (p : P) -> R p
```

```
answer "what year is it?" = 2026      -- an Int, as required
answer "are you awake?"   = True      -- a Bool, as required
```

Each branch returns a value of exactly the type that branch's prompt demands, and the compiler checks it.

Here are the other examples read the same way:

- *Questions and answers*: `P` the questions, `R p` the answers valid for `p`; a program `(p : P) -> R p` answers each question with something that type-checks as an answer to *it*.
- *Theorems and proofs*: `P` the statements, `R p` the proofs of `p`. A program `(p : P) -> R p` is a *proof procedure*: for every statement, it returns a genuine proof of that very statement. (This is not a metaphor — it is literally how proof assistants work.)
- *Data and pointers*: `P` the structures, `R p` the valid positions inside `p`; a program picks, for each structure, a legitimate position within it.

4 Event handlers: handing off interfaces

So far the programmer answers each prompt *directly*. But this is not how real systems — or real organisations — work. Often you cannot answer a high-level prompt yourself; instead you *delegate*. You take the high-level request, translate it into a lower-level one, hand it off, get a lower-level response back, and assemble that into the high-level answer you owe.

This is exactly the relationship between a project manager and a software developer, which is where this note began. The manager receives a high-level requirement — "handle this customer's refund." The manager does not process the refund personally; instead they break it into lower-level tasks for the developer — "look up the order, verify the payment, issue the credit" — and then assemble the developer's results into the answer the original requirement demanded. Control flows *down* (the manager decides which lower-level tasks to request), and results flow back *up* (the developer's outputs are combined into the high-level response).

That pattern is called an **event handler**[1]: it *handles* the high-level interface by invoking a low-level one. Formally it is a **morphism of containers**. Let the manager's interface be the high-level container `(P, R)` and the developer's interface be the low-level container `(Q, T)`. An event handler from `(P, R)` to `(Q, T)` consists of two pieces:

```
u : P -> Q                        -- delegate: high prompt to low prompt
f : (p : P) -> T (u p) -> R p     -- amalgamate: low reply to high reply
```

Read the second line slowly, because it is the heart of the matter. For a given high-level prompt `p`, we fired the low-level prompt `u p` and got back some response of type `T (u p)` — a valid developer-level reply. The function `f` then converts that developer-level reply into a manager-level reply of type `R p` — the answer we actually owed. Control went down through `u`; data came back up through `f`.

This is *small-step* handling: a handler does not have to answer, only to pass the event one level down. The developer's interface `(Q, T)` might in turn be handled by another handler, delegating

[1] This is not quite the *handler* of algebraic effects, though the two are closely related. There, a handler interprets the operations of an effect by giving them meaning in some *interpreting language* — so the language it interprets *into* is already present, fixed in advance. An event handler here is the same delegate-and-amalgamate idea, but with that lower interface made explicit as a container of its own, and with the handler a first-class map *between* two interfaces that one can compose with others. It also differs from the *event handler* of GUI or Node-style programming, which is the *terminal* responder to an event — that is our §3 direct answerer, the special case of handing off to the do-nothing interface. Our event handlers are better thought of as *small-step* handling, described next.

to a still-lower interface, and so on — a tower of hand-offs. It bottoms out only when some level finally answers directly, which (as we are about to see) is itself the special case of handing off to the interface that does nothing. A running system is a stack of these small steps.

An interpreter (high-level language delegated to machine instructions), a device driver (abstract operations delegated to hardware commands), a network adapter (one protocol delegated to another), and a proof-search step (a goal delegated to sub-goals) are all this single construction. Once you see it, delegation is everywhere, and it is always an event handler — a container morphism.

Our bookkeeping assistant is no exception. It does not itself know how to read the books; it handles a high-level `Reconcile` request by delegating to a lower-level *ledger-database* interface — "fetch the postings for this period" — and amalgamating what comes back into the `ReconReport` it owes. The delegate leg turns an accounting task into database queries; the amalgamate leg turns rows of postings into the report. That is one event handler, from the `Books` interface down to the ledger's.

Answering directly is delegating to nobody

There is a satisfying way to see that direct answering (§3) and delegation (§4) are not two different things, but one. Consider the most trivial possible interface: the container

```
I = ( Unit , \_ => Unit )      -- one prompt, one response
```

with a single prompt and a single response — the "do-nothing" interface, where there is nothing to ask and nothing to decide. Now ask: what is an event handler from `(P, R)` into `I`?

Its delegate leg is `u : P -> Unit`, which carries no information (there is only one low-level prompt to fire). Its amalgamate leg is `f : (p : P) -> Unit -> R p`. But a function `Unit -> R p` out of the one-element type is just a choice of element of `R p` — that is, just a value of type `R p`. So the whole handler boils down to: for each `p`, a value of type `R p`. In other words,

```
Cont (P, R) I   =   (p : P) -> R p
```

The direct implementations of §3 are *exactly* the event handlers into the trivial interface. Answering a prompt yourself is the special case of delegation where you hand the work to the interface that does nothing — so you must do it all. Direct response and delegation are the same notion, and the programmer, at every level of a system, is always doing one thing: either answering, or passing the interface along to the level below.

5 Workflows: combining containers

The single-step and single-delegation picture of §3–§4 becomes far more useful once we can *combine* containers into larger ones. Happily, practitioners have already catalogued the combinations that matter. Anthropic's note *Building Effective Agents*[2] identifies a handful of recurring *workflow* patterns — prompt chaining, routing, parallelisation, and so on — and argues that most effective systems are just these simple, composable patterns rather than elaborate frameworks. It is well worth reading first, to see how the same ground is covered from a practitioner's angle.

We are building directly on that catalogue, with one shift of viewpoint. Their workflows are *untyped*: a step is an LLM call, and what flows between steps is unstructured text. And they *separate outputs from inputs* — a workflow has a distinguished starting input and a distinguished final output, with the output sitting literally at the end of the chain. In our typed picture there is no such separation: a container's *response* to one prompt is exactly a *prompt* for the next container, so "output" and "input" are the same kind of thing, and a workflow is just a way of

[2] https://www.anthropic.com/engineering/building-effective-agents

gluing containers along that shared boundary. What this buys us is that each pattern turns out to *be* one of the monoidal structures on containers — so the informal catalogue becomes a small algebra, and the type checker enforces that the pieces fit.

Prompt chaining is sequential composition (<—)

Prompt chaining decomposes a task into a sequence of steps, where each call processes the output of the previous one. That is exactly sequential composition: the response type of one container is the prompt type of the next, and we glue them along that shared boundary. Concretely, given `(P, R)` and `(Q, T)`, their composite `(P, R) < (Q, T)`— is the container

```
-- a prompt is: a first prompt p, plus a way to turn each of its responses
--              into a second-stage prompt
shape   =  (p : P ** R p -> Q)
-- a position is: a response r to p, plus a response to the Q-prompt it chose
pos (p ** v)  =  (r : R p ** T (v r))
```

A chain `c1 < c2 ¡— c3`— is a pipeline: prompt the first, its response prompts the second, and so on. Where Anthropic add a programmatic "gate" between steps to check the process is on track, we would place a container whose prompts are the intermediate results and whose responses are the checked (or rejected) values — a gate is just another link in the <— chain.

Routing is the sum (+)

Routing classifies an input and sends it to one of several specialised followups. The combined interface accepts a prompt for *any* of the branches, and which branch a prompt belongs to determines which responses are valid. That is the sum of containers: prompts are the disjoint union of the branches' prompts, and each prompt carries its own branch's responses. Given `(P, R)` and `(Q, T)`, the sum `(P, R) + (Q, T)` is

```
shape           =  Either P Q          -- a prompt for the left OR the right branch
pos (Left  p)   =  R p                  -- a left prompt keeps its own responses
pos (Right q)   =  T q                  -- a right prompt keeps its own responses
```

Directing general questions, refund requests, and technical-support queries to three different handlers is exactly `general + refund + techSupport`: the router reads which summand a prompt lies in, and dispatches to that handler. The sum is the "choose one branch" combinator.

Parallelisation is the tensor ((x))

Parallelisation runs several subtasks at once and aggregates their outputs. Here you fire a prompt to *every* section and you need a response from *each*, so the responses pair up. That is the tensor: prompts multiply (you supply a prompt to each side), and positions multiply too (you owe a response to each side). Given `(P, R)` and `(Q, T)`, the tensor `(P, R) (x) (Q, T)` is

```
shape         =  (P, Q)              -- a prompt to BOTH sides
pos (p, q)    =  (R p, T q)          -- a reply is a PAIR: one from each side
```

Anthropic split this into *sectioning* (independent subtasks — a guardrail check running alongside the main response) and *voting* (the same task run several times and the results combined). Sectioning of two different tasks is `c (x) d`; voting is the tensor power `c (x) c (x) ... (x) c` followed by an aggregation container that reduces the tuple of responses to one. In both cases the defining move is "run all, collect every reply," which is the tensor.

And the product (x): a pattern the maths predicts

The three patterns above used three of our four monoidal structures — `<—`, `+`, and `(x)`. The fourth, the categorical product `x`, has no named counterpart in the practitioner's catalogue. Rather than force a match, this is worth pausing on: the algebra has *four* ways to combine containers, and the field has so far named only three. The mathematics is predicting a workflow that has not yet been written down.

And we can read off exactly what it is. The product's prompts multiply, like the tensor's, but a response is required to only *one* of the two sides — its positions are a *sum*, not a product. Given `(P, R)` and `(Q, T)`, the product `(P, R) x (Q, T)` is

```
shape          =  (P, Q)                  -- a prompt to both sides (as for the tensor)
pos (p, q)     =  Either (R p) (T q)      -- but a reply to just ONE side
```

Compare the tensor: same shapes `(P, Q)`, but its positions were the *pair* `(R p, T q)` (answer both) where the product's are the *sum* `Either (R p) (T q)` (answer one). So where the tensor says "prompt both sides and collect *every* reply," the product says "prompt both sides, but you owe just *one* reply, from whichever side you choose." That is a real and useful pattern: offer a fast path and a thorough path, run whatever you like, and commit to a single answer from one of them; or present several candidate sub-tasks and require exactly one to be discharged. It sits precisely between routing (`+`, where the *input* already belongs to one branch) and parallelisation (`(x)`, where *every* branch must answer): the product lets both branches be live but demands a single response.

The maths runs ahead of practice

This is a small instance of a familiar pattern in the history of science: the formalism outruns the practitioners. In particle physics the mathematics of symmetry predicted particles — the Ω^-, the Higgs — years before any experiment found them; the equations had a slot, so something had to fill it. Here the container algebra has four monoidal structures, so there are four basic ways to combine interfaces — and the fourth, `x`, names a workflow the catalogue simply has not reached yet. A gap in the table is not an embarrassment; it is a prediction.

The workflow algebra

Workflow	Container structure	Slogan
Prompt chaining	sequential comp. `<—`	output feeds the next input
Routing	sum `+`	choose one branch
Parallelisation	tensor `(x)`	run all, one reply each
Offer-both / one reply	product `x`	both offered, answer one

The two remaining patterns in the note fit the same way, one level up. *Orchestrator–workers* — where a central call dynamically decomposes a task and delegates the parts — is an event handler (§4), a container *morphism* whose low-level interface is built, at runtime, from `<—`, `+` and `(x)`: the orchestrator is the delegate leg choosing how to wire the workers, and the amalgamate leg synthesises their replies. *Evaluator–optimizer* — generate, evaluate, and loop until good — is a *fixpoint*: the same chain `<—` iterated, with a routing `+` gate deciding at each turn whether to stop or go round again. So the entire catalogue is the four monoidal structures, plus morphisms and iteration — and because it is all typed, the compiler checks that the responses of one stage really are valid prompts for the next.

The running example: reconciliation as a workflow

Let us assemble the bookkeeping assistant's handler and watch all four structures appear, each where it belongs. The top-level handler first *routes* on the kind of request — the sum:

```
booksFlow  =  reconcileFlow  +  invoiceFlow  +  taxFlow
```

A request is a `Reconcile`, an `Invoice`, or a `TaxEstimate`; the router reads which and dispatches. Follow the `Reconcile` branch. To reconcile a period we need three pieces of data that are independent of one another, so we fetch them *in parallel* — the tensor:

```
gather  =  bankFeed  (x)  ledger  (x)  outstandingInvoices
```

Every side must answer, and the replies pair up into the full context. With that context, the reconciliation proper is a *pipeline* where each step consumes the previous one's output — sequential composition:

```
pipeline  =  normalise  <|  match  <|  report
```

Normalise the raw lines, match them against the ledger, emit the exceptions report. The whole `Reconcile` branch gathers context, then runs the pipeline:

```
reconcileFlow  =  gather  <|  pipeline
```

And the predicted product `x` appears exactly where §5 said it would. When `report` finds a discrepancy, it may be settled two ways — a *write-off* or an *adjustment* — and the handler commits to only one:

```
settle  =  writeOff  x  adjust      -- both offered; answer exactly one
```

Routing (`+`) chose *which kind of request* we faced; the product (`x`) chooses *which resolution* we commit to — the same "offer several, answer one" shape the algebra predicted, now doing real work.

One handler, the whole algebra

The bookkeeping assistant's reconciliation handler is, in a phrase: route the request (`+`), gather context in parallel (`(x)`), chain the resolution (`<|`), and choose the settlement (`x`). Sum, tensor, sequential composition, and product — the four ways of combining containers — are not four unrelated tricks but the four joints of a single, ordinary workflow. And because every join is typed, the compiler guarantees the pieces fit: a bank-feed reply really is valid input to `match`, and the final report really is the `ReconReport` that `Result (Reconcile p)` demanded back in §3.

6 Where the LLM sits: oracles at the leaves

We can now say precisely where a large language model fits into all of this. An LLM does one thing: it takes a prompt and produces an answer. In our vocabulary that is a *direct answerer* — an event handler into the do-nothing interface. If the LLM is serving the container `(S, P)`, it is a map

```
llm : Cont (S, P) I   =   (s : S) -> P s
```

exactly the shape from the end of §4: given a prompt `s`, return a response in `P s`. An LLM is a **leaf**: it does not delegate further, it just answers.

The shape of an LLM system

The handlers of §4 and the workflows of §5 build a *tree* of typed interfaces. Starting from one high-level interface, each handler simplifies its prompt and hands off to lower-level interfaces — branching, via `<—`, `+` and `(x)`, into a whole collection of smaller sub-prompts. At the *leaves* of that tree sit calls to LLMs, each a direct answerer `(s : S) -> P s`. The replies are then amalgamated back up the tree into the overall answer. A complete system is one big event handler whose oracles are LLMs at the leaves, wrapped in a typed scaffold that decides *what* to ask and *how* to combine what comes back.

This is the division of labour the whole note has been building towards. The *typed part* — the handlers and workflow combinators — carries the structure: it breaks a hard prompt into easier ones, routes and parallelises them, and reassembles the results, all checked by the compiler so that every sub-prompt is answered in its own valid response type. The *LLM part* does the one thing types cannot: supply the actual content at each leaf. The scaffold guarantees the *shape* of the answer; the model supplies its *substance*.

Our bookkeeping assistant shows the split cleanly. The reconciliation workflow of §5 — route, gather, match, report — is all typed scaffold: the compiler guarantees debits are matched against credits and the result is a well-formed `ReconReport`. But at the end we want a human-readable note explaining each discrepancy, and *that* is a job for a language model:

```
explain : (d : Discrepancy) -> Explanation d      -- an LLM leaf
```

a direct answerer sitting at a leaf of the tree. The arithmetic, the matching, and the balancing are done by the typed part, which no LLM should be trusted with; the model is asked only to do what it is genuinely good at — turn a settled discrepancy into plain English. Structure from the types, prose from the model.

One honest caveat completes the picture. A raw LLM is *untyped* — it maps text to text, and nothing about it guarantees its output really lies in `P s`. Presenting it as `(s : S) -> P s` is therefore a *promise*, made good by a typed wrapper around the raw model: constrained decoding, output validation, or a retry loop that rejects ill-typed answers. The interface is the contract; the LLM is the (fallible) oracle behind it; and the wrapper is what turns an untyped text machine into a leaf that honours its type. Everything above the leaves is then exactly as checkable as any other typed program.

Appendices

Two appendices follow. Appendix A is the mathematical backing: a categorical account for the categorically literate reader. Appendix B collects the running bookkeeping example, whose fragments are scattered through the main text, into one complete listing. Both may be skipped by readers who only want the ideas. (The code throughout is written in a dependently typed style; Idris 2 is a language in which it can be made to compile with only minor syntactic adjustment.)

A A categorical take (for the categorically literate)

Motivation

There are two ways into the notion of a container. Each explains a feature the other treats as a coincidence, so it is worth holding both.

First angle: closing the representables under coproducts. We wish to study $[\mathbf{Set}, \mathbf{Set}]$. Two classes of objects are understood at once: the *representables* $\mathbf{y}^A = \mathbf{Set}(A, -)$, together with the Yoneda lemma $[\mathbf{Set}, \mathbf{Set}](\mathbf{y}^A, F) \cong FA$, which computes every natural transformation out of a representable. Representables are closed under little; the first honest closure to take is under *coproducts*. A coproduct of representables is a functor

$$X \longmapsto \coprod_{s \in S} \mathbf{Set}(Ps,\, X),$$

i.e. a set S indexing the summands together with, for each $s \in S$, a representing object Ps. The data is exactly a set-indexed family of sets, which we write

$$(S, P), \qquad S : \mathbf{Set}, \quad P : S \to \mathbf{Set},$$

and call a *container*. We reserve $\lhd$ for the sequential composition of containers (Construction 4) and never overload it for the pair. The functor a container denotes is its *extension*,

$$[\![S, P]\!](X) \;=\; \sum_{s \in S} \mathbf{Set}(Ps, X) \;=\; \sum_{s \in S} X^{Ps}.$$

So containers are the free coproduct completion of the representables inside $[\mathbf{Set}, \mathbf{Set}]$, and Yoneda makes the maps out of each summand computable.

Second angle: containers as objects of interest in their own right. Forget functors for a moment. Read S as a set of *events* and, for each event s, read Ps as the set of *responses* available to that event. The pair (S, P) is then a complete interface specification: which situations may arise, and what one may do in each. The extension $[\![S, P]\!](X) = \sum_s (Ps \to X)$ reads as "choose an event, then label every response with a datum from X." This reading is ubiquitous:

- *Theorem proving.* S is the set of goals; Ps is the set of sub-lemmas that a chosen tactic reduces s to. A finite proof tree is an element of the fixpoint of the extension.
- *Servers.* S is the set of requests a server can handle; Ps is the set of actions it may take in response to request s.
- *Interaction.* S is a set of questions; Ps is the set of admissible answers to question s.

The two angles agree on the objects and, as the next section shows, on the morphisms: the coproduct-completion picture forces exactly the morphisms the event/response picture wants.

The category of containers is $\mathrm{Fam}(\mathbf{Set}^{\mathrm{op}})$

A container (S, P) is precisely an object of $\mathrm{Fam}(\mathbf{Set}^{\mathrm{op}})$: an S-indexed family $(Ps)_{s \in S}$ of objects of $\mathbf{Set}^{\mathrm{op}}$. This fixes the morphisms, since $\mathrm{Fam}(\mathcal{C})$ is the free coproduct completion of $\mathcal{C}$ and its

morphisms are determined by that universal property. Spelled out, a morphism $(S, P) \to (Q, T)$ is a pair

$$u : S \to Q, \qquad \big(f_s : T(u\,s) \to Ps\big)_{s \in S},$$

a map *forward* on shapes and, for each shape, a map *backward* on positions (the position leg is an arrow of $\mathbf{Set}^{\mathrm{op}}$, hence contravariant). This contravariance of positions is the single fact that governs every construction below.

Yoneda into $[\mathbf{Set}, \mathbf{Set}]$. The Yoneda embedding $\mathbf{y} : \mathbf{Set}^{\mathrm{op}} \hookrightarrow [\mathbf{Set}, \mathbf{Set}]$, $A \mapsto \mathbf{Set}(A, -)$, is full and faithful. Extending it along the coproduct completion gives

$$[\![-]\!] : \mathbf{Cont} = \mathrm{Fam}(\mathbf{Set}^{\mathrm{op}}) \longrightarrow [\mathbf{Set}, \mathbf{Set}],$$

which is full and faithful, because $[\mathbf{Set}, \mathbf{Set}]$ has all coproducts and $\mathbf{y}$ is. Concretely,

$$[\mathbf{Set}, \mathbf{Set}]\big([\![S, P]\!], [\![Q, T]\!]\big) \;\cong\; \prod_{s \in S} [\![Q, T]\!](Ps) \;\cong\; \prod_{s \in S} \sum_{q \in Q} \mathbf{Set}\big(Tq,\, Ps\big),$$

the first step by coproducts-out and Yoneda, the second by unfolding the extension. The right-hand side is exactly the container-morphism data: for each shape s, a shape $q = u\,s$ and a map $Tq \to Ps$.

The event-handling reading: delegate and amalgamate. Let (S, P) be a high-level interface and (Q, T) a low-level one; to implement the former over the latter is to give a morphism $(u, f) : (S, P) \to (Q, T)$:

- **Delegate (shapes forward).** A high-level event s is handled by firing the low-level event $u\,s$.
- **Amalgamate (positions backward).** A low-level response $p \in T(u\,s)$ is translated back into the high-level response $f_s(p) \in Ps$ that we owe.

Control flows down, data flows back up: this is why the position leg is contravariant, and why the fibre of Fam is $\mathbf{Set}^{\mathrm{op}}$. An interpreter, a driver, a proof-search reduction, and a protocol adapter are one construction.

Four monoidal structures and two closed structures

We give the general Day convolution first (Construction 4 of the brief), then read off its two special cases. Throughout, the position-level operation determines the container-level operation, with the variance flip doing the work.

Construction 1 (Day convolution of a monoidal product $*$ on $\mathbf{Set}$)**.** Any monoidal product $* : \mathbf{Set} \times \mathbf{Set} \to \mathbf{Set}$, with unit I, lifts to a monoidal product on $\mathbf{Cont}$ by

$$(S, P) \otimes_* (Q, T) \;=\; \big(\,S \times Q,\; (s, q) \mapsto Ps * Tq\,\big), \qquad \text{unit } (1,\; \star \mapsto I).$$

Shapes multiply; positions combine by $*$. This is the Day convolution of $*$: it is $\mathrm{Lan}_{J \times J}$ of $*$ along the singleton embedding $J : \mathbf{Set}^{\mathrm{op}} \hookrightarrow \mathbf{Cont}$, $A \mapsto (1, \star \mapsto A)$, and it inherits associativity, symmetry, and unit from $*$. Being a left Kan extension in each variable, $\otimes_*$ preserves colimits in each variable and hence has a right adjoint: a *internal hom* $\multimap_*$ with

$$\mathbf{Cont}\big((S, P) \otimes_* (Q, T),\; (R, U)\big) \;\cong\; \mathbf{Cont}\big((S, P),\; (Q, T) \multimap_* (R, U)\big).$$

Construction 2 (Product: $* = +$ on positions)**.** Take $* = +$, the coproduct of $\mathbf{Set}$. Because positions are contravariant, a *product* of shapes pairs with a *sum* of positions:

$$(S, P) \times (Q, T) \;=\; \big(\,S \times Q,\; (s, q) \mapsto Ps + Tq\,\big), \qquad \text{unit } (1,\; \star \mapsto \varnothing).$$

This is the *categorical product* in $\mathbf{Cont}$: its extension is the pointwise product of extensions, $[\![S, P \times Q, T]\!](X) = \sum_{(s,q)} X^{Ps+Tq} = \big(\sum_s X^{Ps}\big) \times \big(\sum_q X^{Tq}\big)$, using $X^{A+B} = X^A \times X^B$. The Day internal hom for $+$ is the corresponding exponential.

Construction 3 (Tensor: $* = \times$ on positions)**.** Take $* = \times$, the product of **Set**. Then

$$(S,P) \boxtimes (Q,T) \;=\; \big(S \times Q,\ (s,q) \mapsto Ps \times Tq\big), \qquad \text{unit } (1,\ \star \mapsto 1),$$

the *tensor* of containers – Day convolution of $\times$, distinct from the categorical product above. Its internal hom $\multimap_\times$ makes $(\mathbf{Cont}, \boxtimes)$ closed; this is the symmetric monoidal closed structure under which containers model parallel, non-interfering interaction.

Construction 4 (Sequential composition $\lhd$)**.** The fourth structure is not a Day convolution but composition of extensions: $[\![(S,P) \lhd (S',P')]\!] = [\![S,P]\!] \circ [\![S',P']\!]$. Explicitly, $(S,P) \lhd (S',P') = (Q,T)$ where

$$Q \;=\; \sum_{s \in S} \big(Ps \to S'\big), \qquad T\,(s,v) \;=\; \sum_{p \in Ps} P'\big(v\,p\big) \quad \text{for } v : Ps \to S'.$$

A shape of the composite is a downstairs shape s together with a choice v of an upstairs shape at each of its positions; a position of the composite is a downstairs position p paired with an upstairs position of the shape $v\,p$ sitting over it. The structure is monoidal with unit the identity container $(1, \star \mapsto 1)$. Monoids for $\lhd$ are monads (free algebras are the W-types of §A.1); comonoids for $\lhd$ are the directed containers, i.e. small categories.

Lemma 1 (Preservation)**.** *The extension* $[\![-]\!] : \mathbf{Cont} \to [\mathbf{Set}, \mathbf{Set}]$ *preserves all four monoidal structures and both closed structures. Explicitly, up to natural isomorphism,*

$$[\![A \times B]\!] \cong [\![A]\!] \times [\![B]\!], \quad [\![A \boxtimes B]\!] \cong [\![A]\!] \otimes [\![B]\!], \quad [\![A \lhd B]\!] \cong [\![A]\!] \circ [\![B]\!],$$

each carrying units to units; and $[\![-]\!]$ *takes the internal homs* $\multimap_+$ *and* $\multimap_\times$ *to the corresponding right adjoints in* $[\mathbf{Set}, \mathbf{Set}]$*. Hence* $[\![-]\!]$ *is a full and faithful, coproduct-preserving, strong monoidal closed embedding for each structure simultaneously.*

Proof sketch. $[\![-]\!]$ is full and faithful (§2) and preserves coproducts by construction. The isomorphisms for $\times$ and $\boxtimes$ are the exponential law $X^{A+B} \cong X^A \times X^B$ and the Fubini rule for $\sum/\times$; for $\lhd$ it is the definition. Each Day tensor is a left Kan extension, so its right adjoint exists and is a colimit $[\![-]\!]$ preserves, hence carried to the matching right adjoint on $[\mathbf{Set}, \mathbf{Set}]$. □

Co-closure by left Kan extension. A different use of Lan gives a construction *on containers* rather than on their tensors. For a functor $F : \mathbf{Set} \to \mathbf{Set}$, precomposition $\mathrm{Lan}_F(-) = (-) \circ F$ sends representables to representables ($\mathbf{y}^A \circ F \cong \mathbf{y}^{FA}$) and hence extensions of containers to extensions of containers. Reflecting through the full and faithful $[\![-]\!]$ gives an operation on containers themselves,

$$(S,P) \;\longmapsto\; (S,\ F \circ P),$$

acting on positions only, shapes fixed – the arity-1 case of the Day recipe ($n = 1$). It exhibits the sequence and other free structures as *co-closed*: dually to the right-adjoint internal homs of the Day tensors, these left adjoints $(S,P) \mapsto (S, F \circ P)$ supply the co-hom side, and $[\![-]\!]$ preserves this too.

Conclusion

Containers are $\mathrm{Fam}(\mathbf{Set}^{\mathrm{op}})$: their morphisms delegate on shapes and amalgamate on positions, and Yoneda embeds them full-and-faithfully into $[\mathbf{Set}, \mathbf{Set}]$, with $[\![-]\!]$ preserving the product ($+$ on positions), the tensor ($\times$ on positions), sequential composition $\lhd$, the generic Day convolution, and both associated internal homs – while $(S,P) \mapsto (S, F \circ P)$ is the co-closed, position-only reflection of Lan_F. The upshot is a small piece of data carrying an extremely rich categorical structure: two closed symmetric monoidal products, a composition monoidal product whose monoids and comonoids are exactly monads and small categories, a coproduct completion, and a battery of adjoints – all of it computable at the level of the pair (S,P) and all of it preserved by a single full and faithful embedding.

That combination – concrete data, structural richness – is why the same object recurs across so many settings. In *theorem proving* a container is a rule signature (goals to sub-goals), its $\triangleleft$-monad proof search; in *workflows* a task interface (jobs to steps), with $\triangleleft$ sequencing and $\boxtimes$ parallelism; in *agentic AI* an event/response boundary (observations and available actions), where container morphisms are the delegate-and-amalgamate implementations of a high-level policy over a low-level API; and in *data types* the shapes-and-positions presentation of a datatype, whose $\triangleleft$-monoids and W-types are the inductive families. One vocabulary – shapes fire, positions respond, control flows down and data flows back up – serves all four.

The natural next step is to let containers *interact*. Systems of interacting containers are presheaves (or sheaves) valued in **Cont**, i.e. functors $[\mathcal{C}^{\mathrm{op}}, \mathbf{Cont}]$ over a base $\mathcal{C}$ of sites, locations, or agents: each object of $\mathcal{C}$ carries a container, and restriction maps are container morphisms wiring one interface into another, with a gluing condition where local interfaces must agree. This is the setting Eric and Neil are pursuing, and it is where the closed and co-closed structure above earns its keep: the tensor $\boxtimes$ composes non-interfering local systems, the internal homs describe how one interacting system acts on another, and the co-closure $(S, P) \mapsto (S, F \circ P)$ lets a functor F act fibrewise across the whole diagram at once.

Crucially, $\mathbf{Cont} = \mathrm{Fam}(\mathbf{Set}^{\mathrm{op}})$ is both *complete and cocomplete*: coproducts are disjoint unions of shapes, $(S, P) + (Q, T) = (S + Q, [P, T])$; products multiply shapes and, dually, sum positions, $\prod_i (S_i, P_i) = \left(\prod_i S_i,\ (s_i)_i \mapsto \sum_i P_i(s_i)\right)$ (the Construction 2 pattern at arbitrary arity); and the remaining limits and colimits are computed from those together with the (co)completeness of $\mathbf{Set}^{\mathrm{op}}$. Because the codomain **Cont** has all small limits and colimits, presheaves valued in it can be *left and right Kan extended* along any functor $g : \mathcal{C} \to \mathcal{D}$ of base categories:

$$g_! \dashv g^* \dashv g_* \ :\ [\mathcal{C}^{\mathrm{op}}, \mathbf{Cont}] \longrightarrow [\mathcal{D}^{\mathrm{op}}, \mathbf{Cont}],$$

with g^* restriction, $g_! = \mathrm{Lan}_{g^{\mathrm{op}}}$ the left Kan extension (pointwise a colimit in **Cont**), and $g_* = \mathrm{Ran}_{g^{\mathrm{op}}}$ the right (pointwise a limit). This is a *change of base topology*: a map between underlying sites transports an entire interacting system to a new site, colimit-style or limit-style, gluing or cofree. The theory of a single container is the object language; $[\mathcal{C}^{\mathrm{op}}, \mathbf{Cont}]$, with its Kan-extension change of base, is the theory of how many of them fit together.

B The bookkeeping assistant, assembled

The running example of §2–§6 was introduced a fragment at a time. For completeness, here it is in one piece: the double-entry invariant, the `Books` interface, the reconciliation workflow with all four monoidal structures, and the LLM leaf. It is written in a dependently typed style (Idris 2 with minor adjustment); it is meant to be read, not run, though it is close to compilable.

The container vocabulary

The four combinators, defined on containers exactly as in §5:

```
record Cont where
  constructor MkCont
  Shape : Type
  Pos   : Shape -> Type

sumC, prodC, tensorC, seqC : Cont -> Cont -> Cont
sumC c d    = MkCont (Either c.Shape d.Shape)
  (\s => case s of Left cs => c.Pos cs; Right ds => d.Pos ds)    -- routing +
prodC c d   = MkCont (c.Shape, d.Shape)
  (\sq => Either (c.Pos (fst sq)) (d.Pos (snd sq)))              -- product x : answer
    one
tensorC c d = MkCont (c.Shape, d.Shape)
```

```
  (\sq => (c.Pos (fst sq), d.Pos (snd sq)))                    -- tensor (x) : answer
    both
seqC c d    = MkCont (s : c.Shape ** (c.Pos s -> d.Shape))
  (\sv => (p : c.Pos (fst sv) ** d.Pos (snd sv p)))            -- chaining <|

infixr 5 <|
(<|) : Cont -> Cont -> Cont
(<|) = seqC
```

Double entry as a dependent type (§2)

```
record Posting where
  constructor MkPosting
  account : String
  amount  : Integer            -- in pennies; debit > 0, credit < 0

netAmount : List Posting -> Integer
netAmount []        = 0
netAmount (p :: ps) = p.amount + netAmount ps

Balanced : List Posting -> Type
Balanced ps = (netAmount ps = 0)          -- debits = credits, as a type

BalancedEntry : Type
BalancedEntry = (ps : List Posting ** Balanced ps)   -- postings PLUS a proof

-- a 100.00 sale: debit cash, credit revenue -- provably balanced
saleEntry : BalancedEntry
saleEntry = ([ MkPosting "cash" 10000, MkPosting "revenue" (-10000) ] ** Refl)
--   change either amount and `Refl` no longer typechecks: the books MUST balance
```

The interface `Books = (Request, Result)` (§3)

```
data Period   = MkPeriod String
data Customer = MkCustomer String
data Year     = MkYear Nat

data ReconReport  = MkReconReport (List Posting) (List Posting)  -- matched / unmatched
data ValidInvoice = MkValidInvoice BalancedEntry
data TaxFigure    = MkTaxFigure Integer

data Request = Reconcile    Period
             | RaiseInvoice Customer
             | TaxEstimate  Year

Result : Request -> Type                    -- the response type depends on the request
Result (Reconcile    _) = ReconReport
Result (RaiseInvoice _) = ValidInvoice
Result (TaxEstimate  _) = TaxFigure

Books : Cont
Books = MkCont Request Result

-- to implement the assistant directly is to give a dependent function
Assistant : Type
```

```
Assistant = (r : Request) -> Result r        -- exactly  Cont Books I
```

The reconciliation workflow (§5): `+`, `(x)`, `<—`, `x`

```
-- lower-level interfaces the reconciliation delegates to
bankFeed, ledgerQ, outstanding : Cont
bankFeed    = MkCont Period (\_ => List Posting)
ledgerQ     = MkCont Period (\_ => List Posting)
outstanding = MkCont Period (\_ => List Posting)

-- (x) PARALLEL: gather three independent sources at once; a reply from EACH
gather : Cont
gather = bankFeed `tensorC` ledgerQ `tensorC` outstanding

-- <| CHAIN: each stage's response is the next stage's prompt
normalise, match, report : Cont
normalise = MkCont (List Posting) (\_ => List Posting)
match     = MkCont (List Posting) (\_ => (List Posting, List Posting))
report    = MkCont (List Posting, List Posting) (\_ => ReconReport)

pipeline : Cont
pipeline = normalise <| match <| report

-- x PRODUCT (the predicted pattern): settle a discrepancy one way OR the other
writeOff, adjust : Cont
writeOff = MkCont Posting (\_ => BalancedEntry)
adjust   = MkCont Posting (\_ => BalancedEntry)

settle : Cont
settle = writeOff `prodC` adjust             -- both offered; answer exactly one

-- the Reconcile branch: gather context, then run the pipeline
reconcileFlow : Cont
reconcileFlow = gather <| pipeline

-- + ROUTING at the top: dispatch by request kind
invoiceFlow, taxFlow : Cont
invoiceFlow = MkCont Customer (\_ => ValidInvoice)
taxFlow     = MkCont Year     (\_ => TaxFigure)

booksFlow : Cont
booksFlow = reconcileFlow `sumC` invoiceFlow `sumC` taxFlow
--   +   routes the request           (booksFlow)
--   (x) gathers context in parallel  (gather)
--   <|  chains the resolution        (pipeline)
--   x   chooses the settlement       (settle)
```

The LLM leaf (§6)

```
data Discrepancy = MkDiscrepancy Posting Posting
data Explanation = MkExplanation String

explainC : Cont
explainC = MkCont Discrepancy (\_ => Explanation)
```

```
-- an LLM serving explainC is a direct answerer -- a leaf:
--     explain : (d : Discrepancy) -> Explanation      (i.e. Cont explainC I)
-- the typed scaffold does the arithmetic; the model writes the plain-English note.
```